

## 1. Block-Level Elements

A **block-level element** usually starts on a **new line** and takes up the available width of its parent/container.

### Example

```
<h1>Welcome</h1>
```

```
<p>Hello Students</p>
```

```
<div>This is a div</div>
```

They appear like:

Welcome

Hello Students

This is a **div**

Each element starts on a new line.

### Common Block-Level Elements

```
<h1> to <h6>
```

```
<p>
```

```
<div>
```

```
<section>
```

```
<article>
```

```
<header>
```

```
<footer>
```

```
<nav>
```

```
<main>
```

```
<aside>
```

```
<ul>
```

```
<ol>
```

```
<li>
```

```
<table>
```

```
<form>
```

### Example

`<h1>My Website</h1>`

`<p>Welcome to my website.</p>`

`<div>About My Website</div>`

Each one starts on a new line.

### Easy Definition

**Block-level elements start on a new line and generally occupy the available width.**

---

## 2. Inline Elements

An **inline element does not normally start on a new line.**

It takes only the space required by its content.

### Example

`<span>Hello</span>`

`<span>Students</span>`

`<span>Welcome</span>`

Output:

Hello Students Welcome

They can appear on the **same line.**

### Common Inline Elements

`<span>`

`<a>`

`<strong>`

`<b>`

`<em>`

`<i>`

`<u>`

`<small>`

`<label>`

**<code>**

**<mark>**

**<sub>**

**<sup>**

### Example

**<p>**

I am learning

**<strong>HTML</strong>**

and

**<strong>CSS</strong>.**

**</p>**

Here, **<strong>** stays within the same line.

### Easy Definition

**Inline elements stay in the same line and generally occupy only the space required by their content.**

---

### Block vs Inline

| Block-Level                                        | Inline                                   |
|----------------------------------------------------|------------------------------------------|
| Starts on a new line                               | Does not normally start on a new line    |
| Generally takes available width                    | Takes only required width                |
| Can contain other elements depending on HTML rules | Usually used for small pieces of content |
| Example: <b>&lt;div&gt;</b>                        | Example: <b>&lt;span&gt;</b>             |
| Example: <b>&lt;p&gt;</b>                          | Example: <b>&lt;a&gt;</b>                |
| Example: <b>&lt;section&gt;</b>                    | Example: <b>&lt;strong&gt;</b>           |

### Easy Memory Trick

BLOCK

↓

New **Line**

INLINE

↓

Same **Line**

---

### 3. Semantic Elements

**Semantic means "meaningful."**

Semantic HTML elements clearly tell the browser and developer **what the content represents**.

Example:

**<header>**

Website **Header**

**</header>**

**<nav>**

Navigation

**</nav>**

**<main>**

Main Content

**</main>**

**<footer>**

Website **Footer**

**</footer>**

Just by looking at the tags, we can understand their purpose.

---

## Common Semantic Elements

**<header>**

**<nav>**

**<main>**

**<section>**

**<article>**

**<aside>**

**<footer>**

**<figure>**

**<figcaption>**

**<details>**

**<summary>**

## Example

**<header>**

**<h1>My Website</h1>**

**</header>**

**<nav>**

**<a href="#">Home</a>**

**<a href="#">About</a>**

**</nav>**

**<main>**

**<section>**

**<h2>About Me</h2>**

**<p>I am a Full Stack Developer.</p>**

`</section>`

`</main>`

`<footer>`

`<p>Copyright 2026</p>`

`</footer>`

This structure is easy for developers and browsers to understand.

---

## Why Use Semantic HTML?

Semantic HTML provides:

### 1. Better Readability

Developers can easily understand the structure.

`<header>`

`<nav>`

`<main>`

`<footer>`

is much clearer than using only `<div>` everywhere.

### 2. Better Accessibility

Screen readers can better understand the page structure.

### 3. Better SEO

Search engines can better understand the meaning and structure of webpage content.

### 4. Better Maintainability

Large projects become easier to read and maintain.

---

## 4. Non-Semantic Elements

Non-semantic elements **do not clearly describe the meaning of their content**.

The two most common examples are:

`<div>`

**<span>**

**<div>**

<div> is a generic **block-level container**.

Example:

**<div>**

**<h1>**My Website**</h1>**

**<p>**Welcome **to** my website.**</p>**

**</div>**

The <div> itself doesn't tell us what the content means.

It simply acts as a container.

---

**<span>**

<span> is a generic **inline container**.

Example:

**<p>**

My name is

**<span>**Arun**</span>**.

**</p>**

<span> doesn't tell us that "Arun" is a name.

It simply groups the text.

---

### Semantic vs Non-Semantic

| Semantic                  | Non-Semantic              |
|---------------------------|---------------------------|
| Has meaningful name       | Generic name              |
| Clearly describes purpose | Does not describe purpose |
| Better page structure     | Mainly used as containers |

|                     |                                  |
|---------------------|----------------------------------|
| Helps accessibility | Less meaningful by itself        |
| Helps SEO           | Doesn't provide semantic meaning |
| <header>            | <div>                            |
| <nav>               | <div>                            |
| <article>           | <div>                            |
| <section>           | <div>                            |
| <footer>            | <span>                           |

---

### Very Important Difference

Don't confuse these two classifications.

#### Block vs Inline

This describes **how an element behaves in the layout**.

Block

Inline

#### Semantic vs Non-Semantic

This describes **whether an element has meaningful purpose**.

Semantic

Non-Semantic

So an element can be:

Block + Semantic

Inline + Semantic

Block + Non-Semantic

Inline + Non-Semantic

#### Examples



| Element   | Layout | Meaning      |
|-----------|--------|--------------|
| <div>     | Block  | Non-semantic |
| <span>    | Inline | Non-semantic |
| <header>  | Block  | Semantic     |
| <section> | Block  | Semantic     |
| <article> | Block  | Semantic     |
| <nav>     | Block  | Semantic     |
| <footer>  | Block  | Semantic     |
| <a>       | Inline | Semantic     |
| <strong>  | Inline | Semantic     |

## Real-Time Website Example

Imagine an e-commerce website.

Instead of:

```
<div>
```

```
  <div>Products</div>
```

```
  <div>Product information</div>
```

```
  <div>Related products</div>
```

```
  <div>Copyright</div>
```

```
</div>
```

We can use semantic HTML:

```
<header>
```

```
  <h1>My Store</h1>
```

```
</header>
```

**<nav>**

**<a href="#">Home</a>**

**<a href="#">Products</a>**

**<a href="#">Contact</a>**

**</nav>**

**<main>**

**<section>**

**<h2>Products</h2>**

**<article>**

**<h3>Laptop</h3>**

**<p>Powerful laptop for developers.</p>**

**</article>**

**</section>**

**</main>**

**<footer>**

**<p>Copyright 2026</p>**

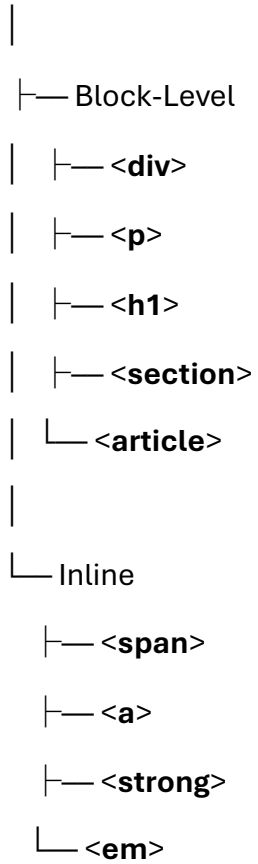
**</footer>**

Now the structure itself explains the webpage.

---

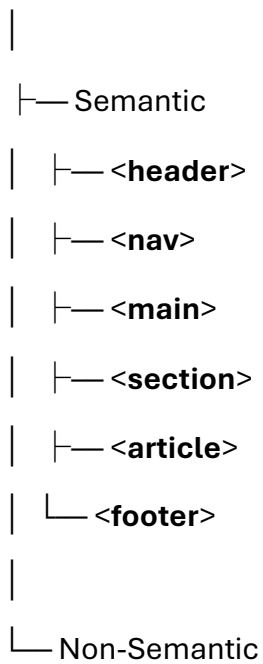
## Quick Revision

### HTML Elements



And separately:

### HTML Elements



|— <div>

└— <span>

## Interview One-Liners

### Block-level element:

A block-level element normally starts on a new line and occupies the available width.

### Inline element:

An inline element normally stays in the same line and occupies only the space required by its content.

### Semantic element:

A semantic element clearly describes the meaning and purpose of its content.

### Non-semantic element:

A non-semantic element does not describe the meaning of its content and is mainly used as a generic container.

### Most important examples to remember:

<div> → Block + Non-Semantic

<span> → Inline + Non-Semantic

<header> → Block + Semantic

<nav> → Block + Semantic

<section> → Block + Semantic

<article> → Block + Semantic

<footer> → Block + Semantic

<a> → Inline + Semantic

<strong> → Inline + Semantic

<em> → Inline + Semantic